Πανεπιστήμιο Κρήτης –Τμήμα Επιστήμης Υπολογιστών

ΗΥ252– Αντικειμενοστρεφής Προγραμματισμός

Διδάσκων: Ι. Τζίτζικας

Χειμερινό Εξάμηνο 2020-2021

Ioannis Kapetanakis

CSD4641

Εισαγωγή

Περιεχόμενα

1. Εισαγωγή 2

2. Η Σχεδίαση και οι Κλάσεις του Πακέτου Model 2

3. Η Σχεδίαση και οι Κλάσεις του Πακέτου Controller 9

4. Η Σχεδίαση και οι Κλάσεις του Πακέτου View 11

5. Η Αλληλεπίδραση μεταξύ των κλάσεων – Διαγράμματα UML 13

6. Λειτουργικότητα (Β Φάση) 14

7. Συμπεράσματα 14

1. Εισαγωγή

Το project αυτό αναπαριστά το επιτραπέζιο παιχνίδι “Sorry”.

Για την υλοποίηση του παιχνιδιού, ο κώδικας έχει οργανωθεί με βάση το αρχιτεκτονικό μοτίβο MVC (Model – View – Controller) και έχει χωριστεί σε 3 κομμάτια.

Το πρώτο κομμάτι αναφέρεται στο Model, δηλαδή στο κομμάτι του κώδικα που είναι υπέυθυνο για την διαχείριση και την εφαρμογή των δεδομένων.

Το δεύτερο κομμάτι αναφέρεται στο View, δηλαδή στη γραφική διεπαφή η οποία ενημερώνεται κατάλληλα βάσει των ενεργειών του χρήση.

Το τελευταίο κομμάτι αναφέρεται στον Controller. Ο Controller είναι υπεύθυνος για την επικοινωνία του χρήστη με το πρόγραμμα. Με λίγα λόγια, είναι ο συνδετικός κρίκος μεταξύ του Model και του View.

2. Η Σχεδίαση και οι Κλάσεις του Πακέτου Model

Το πακέτο Model περιέχει ένα Interface Card το οποίο γίνεται implement από την abstract κλάση NumberCard η οποία με τη σειρά της χρησιμοποιείται για την υλοποίηση των καρτών του παιχνιδιού.

Επίσης μέσα στο πακέτο υπάρχουν οι κλάσεις Deck, Pawn και Player οι οποίες αναπαριστούν τις οντότητες τράπουλα, πιόνι και παίκτης αντίστοιχα καθώς και ένα Enum Position το οποίο επιτρέπει να αντλήσουμε πληροφορία για τη θέση του πιονιού.

Τέλος, έχω δημιουργήσει κλάσεις οι οποίες χρησιμοποιούνται για την αναπαράσταση των τετραγώνων του ταμπλού. Έχω δημιουργήσει 2 abstract κλάσεις. Η μία είναι η SlideSquare την οποία θα αξιοποιήσω για την απεικόνιση των τετραγώνων που έχουν την ιδιότητα του Slide και εξειδικεύει τις κλάσεις StartSlideSquare, InternalSlideSquare και EndSlideSquare. Η άλλη είναι η Square την οποία θα αξιοποιήσω για την αναπαράσταση των όλων των τετραγώνων του ταμπλού, μαζί και των Slide Squares. Η κλάση Square εξειδικεύει τις κλάσεις StartSquare, SimpleSquare, SafetyZoneSquare, HomeSquare, SlideSquare, StartSlideSquare, InternalSlideSquare και EndSlideSquare.

Παρακάτω θα αναφερθώ σε κάθε κλάση του Model ξεχωριστά και θα επεξηγήσω τις μεθόδους τους.

Interface Card

Public String getValue() - returns the value of the card

Public ImageIcon getImage() - returns the image of the card

Class NumberCard

NumberCard(String value) - Initializes the value of the card

Public String getValue() - Returns the value of the card

Public abstract ImageIcon getImage() - Returns the image of the card

Public void setAvailableCards(int num) - Changes the number of available two cards

Public void getAvailableCards() - Returns the number of available two cards

Public ImageIcon getImage() - Returns the image of a two card

Class SorryCard

SorryCard() - Initializes the value and the number of available sorry cards

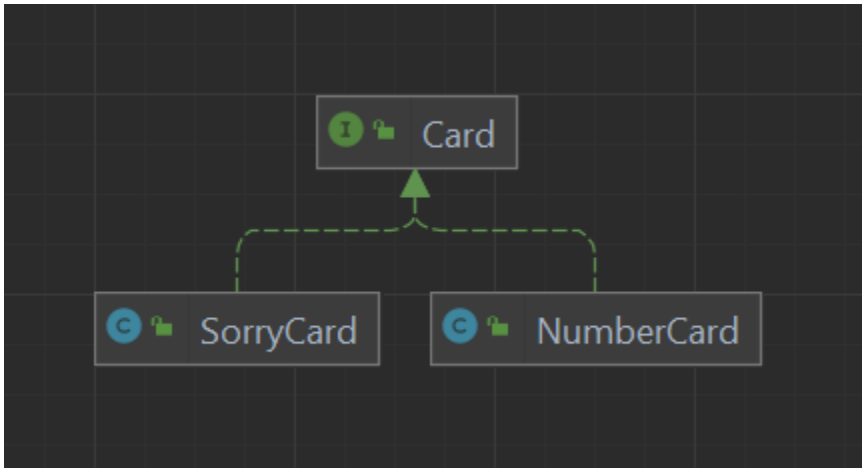
Public void setAvailableCards(int num) - Changes the number of available sorry cards

Public void getAvailableCards() - Returns the number of available sorry cards

Public String getValue() - Returns the value of a sorry card

Public ImageIcon getImage() - Returns the image of a sorry card

UML του Interface Card και των κλασεων που το υλοποιούν:



Class Deck

Deck() - Initializes the deck

Public void init_cards() - initializes the cards of the deck (adds the cards on arraylist of unused cards)

Public void remove_cards() - remove a card from the deck

Public boolean deckIsEmpty() - Check if the deck is empty

Public void refill_deck() - Reinitialise the deck of the cards if its empty

Public int deckSize() - Returns the number of the remained cards of the deck

Public Card getCard(int index) - Returns the card of a specific index on the arraylist

Public ArrayList<Card> getUnusedCards – Returns an arraylist that contains the cards that have not picked yet

Public ArrayList<Card> getUsedCards – Returns an arraylist that contains the cards that have picked

Public void setTotalCards(int totalCards) - Changes the number of the total cards of the deck

Public void getTotalCards() - Returns the number of the total cards of the deck

Class Pawn

Pawn(Player player, int id, String name) - Initializes the holder, the color, the position and the name of the pawn

Public void setPosition(Position pos) - Changes the position of the pawn

Public Position getPosition() - Returns the position of the pawn

Public Color getColor() - Returns the color of the pawn

Public ImageIcon getImage() -Returns the image of the pawn

Public String getName() - Returns the name of the pawn

Public int[] getXY – Returns the coordinates of the pawn on the board

Public void setX(int X) - Changes the x coordinate of the pawn on the board

Public void setY(int X) - Changes the y coordinate of the pawn on the board

Public void setPlayed(boolean bool) - Sets true if a pawn has moved, else false

Public boolean hasPlayed() - returns true if the pawn has moved, else false

Enum Position

Start – In case that the pawn is on start square

Slide – In case that the pawn is on a slide square

SafetyZone – In case that the pawn is on a Safety Zone Square

Home – In case that the pawn is on a Home square

SimplePosition – In case that the pawn is on a Simple square

Class Player

Player(String name, int id, Color color) - Initializes the name, the id and the color of the player

Public void setName(String name) - Changes the name of the player

Public void setColor(Color color) – Changes the color of the player

Public void setId(int id) - Changes the id of the player

Public void setPawn1(Pawn pawn1) - Initializes the first pawn of the player

Public void setPawn2(Pawn pawn2) - Initializes the second pawn of the player

Public String getName() - Returns the name of the player

Public int getId() - Returns the id of the player

Public Color getColor() - Returns the color of the player

Public Pawn getPawn1() - Returns the first pawn of the player

Public Pawn getPawn2() - Returns the second pawn of the player

Public boolen has_finished() - Checks if the player has moved both of his pawns to the home

Public boolean getFoldPressed() - Checks if fold button is pressed

Public boolean setFoldPressed() - Represents the press and reinitialize

Public boolean has_played() - Checks if player completed its moves

Abstract Class Square

Δεν περιέχει κάποια μέθοδο, προς το παρόν

Abstract Class SlideSquare

SlideSquare(Color color) - Initializes the color of the slide square

Abstract public ImageIcon getImage() - Returns the image of the Slide Square

Public void setColor(Color color) - Changes the color of the Slide Square

Public Color getColor() - Returns the color of the Square

Class StartSlideSquare

StartSlideSquare(Color color) - Initializes the color of the start slide square

Public ImageIcon getImage – Returns the image of the start slide square

Class InternalSlideSquare

InternalSlideSquare(Color color) - Initializes the color of the Internal Slide Square

Public ImageIcon getImage() - Returns the image of the Internal Slide Square

Class EndSlideSquare

EndSlideSquare(Color color) - Initializes the color of the end Slide Square

Public ImageIcon getImage() - Returns the image of the end Slide Square

Class HomeSquare

HomeSquare(Color color) - Initializes a home square and its color

Public void setColor(Color color) - Changes the color of the Home Square

Public Color getColor() - Returns the color of the home square

Class SafetyZoneSquare

SafetyZoneSquare(Color color) - Initializes a safety zone square and its color

Public void setColor(Color color) - Changes the color of a safety zone Square

Public Color getColor() - Returns the color of a safety zone square

Class SimpleSquare

Δεν περιέχει κάποια μέθοδο προς το παρόν

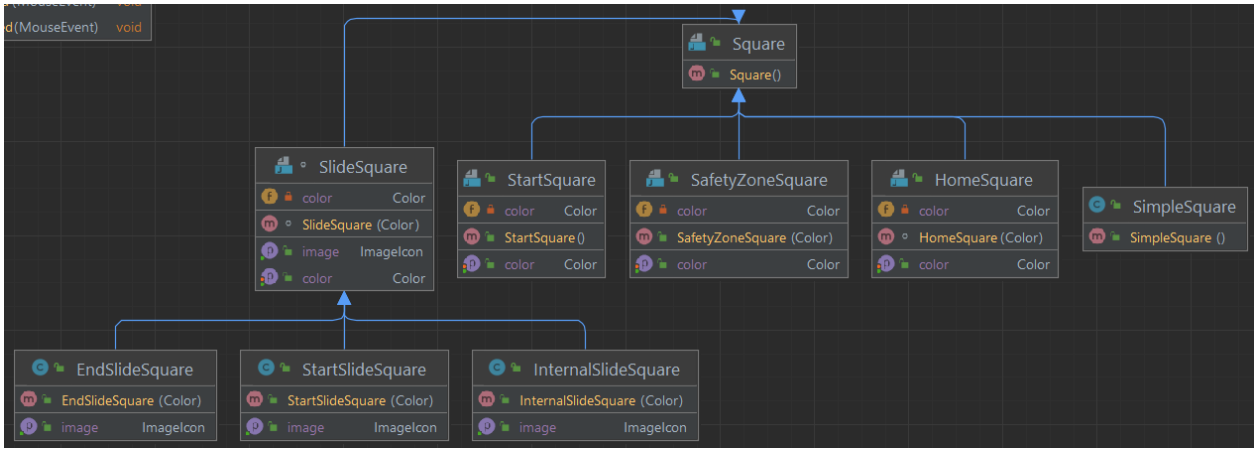
Class StartSquare

StartSquare(Color color) - Initializes a start square and its color

Public void setColor(Color color) - Changes the color of a start Square

Public Color getColor() - Returns the color of a start square

UML της abstract κλάσης Square και των υποκλάσεών της



Για την υλοποίηση της 2ης φάσης, διέγραψα τις κλάσεις που αναπαριστούσαν κάθε κάρτα ξεχωριστά (με εξαίρεση την κάρτα Sorry) και αντί αυτού, θεώρησα καλύτερη ιδέα να χρησιμοποιήσω την κλάση NumberCard απλώς με διαφορετικές παραμέτρους στον Constructor.

Επίσης, στην κλάση Pawn πρόσθεσα τις συναρτήσεις getName, getXY, setX, setY, setPlayed, hasPlayed.

Ο λόγος που έκανα αυτές τις αλλαγές είναι για να γίνει η υλοποίηση πιο εύκολη, καθώς και για να μειωθεί η πολυπλοκότητα του κώδικα.

3. Η Σχεδίαση και οι Κλάσεις του Πακέτου Controller

Παρακάτω θα αναφερθώ στην κλάση Controller του Controller και θα επεξηγήσω τις μεθόδους που περιέχει.

Class Controller

Controller() - Initializes the program's Controller as well as some useful objects for its operation such as players, pawns, decks and cards

Public int getTurn() - Returns the turn

Public void changeTurn() - Changes the turn between the players

Public void setLastCardValue(String Value) - Changes the value of the last card that is picked

Public String getLastCardValue() - Returns the value of the last card that is picked

Public String getRandomCard() - returns a random card from the deck

Public Card getRandomCard() - Returns a random card from the deck

Public boolean FoldPressed() - Checks if the fold button is pressed

Public boolean CanFold(Player player) - Checks if the player can fold

Publuc boolean gameFinished() - Checks if the game has finished

Public void setDeck() - Initializes the deck of the controller

Public void init_board() - Initializes the board

Public void MovePawn() - Implements the movement of the pawns

Public Pawn getPawn11() - Returns red player's first pawn

Public Pawn getPawn12() - Returns red player's second pawn

Public Pawn getPawn21() - Returns yellow player's first pawn

Public Pawn getPawn22() - Returns yellow player's second pawn

Public Position getPosition(Pawn pawn) - returns a specific pawn's position type

Public Position getPosition(Pawn pawn, Position position) - Sets to a pawn a new position type

Public int numOfCards() - returns the remained number of cards

Public ImageIcon LastCardImage() - returns the image of the last card that is drawn by a player

Public int CalculateMove(String value, Pawn pawn) - Calculates the new coordinates of the pawn on the board

Public int[][] getBoard() - returns the board of the game

Public boolean isRoundEnded() - returns true if the current round has ended, else false

Public void setRoundEnded(boolean round_ended) - If a round is over, then set true, else set false

Public Pawn getPawnOnIndex(int i, int j) - Returns pawn of index i,j

Public void moveToStart_onBoard(Pawn pawn) - returns the pawn back to start position on the board

Public player returnWinner() - returns the winner of the game

Από τον Controller διέγραψα τις κλάσεις CheckMove, init_pawns(), checkIfPawnExists() καθώς δεν τις χρησιμοποίησα εν τέλει σε κάποιο σημείο του project

Επίσης, σε σχέση με την πρώτη φάση πρόσθεσα τις συναρτήσεις getPawn11, getPawn12, getPawn21, getPawn22, getPosition, setPosition, numOfCards, LastCardImage, CalculateMove, getBoard, isRoundEnded, setRoundEnded, getPawnOnIndex, moveToStart_onBoard, returnWinner που με βοήθησαν στην υλοποίηση του κώδικα.

4. Η Σχεδίαση και οι Κλάσεις του Πακέτου View

Παρακάτω θα αναφερθώ στην κλάση GUI του View και θα επεξηγήσω τις μεθόδους που περιέχει.

Class GUI

GUI – Constructor who initializes the user interface

Public void init_components() - initializes many components on the board, like background image, cardlistener, game panel, background image and buttons

Public void init_buttons() - initializes the buttons on the board

Public string turn_function(int turn) - returns the turn as a string

Public void cardSeven_Movement(JButton but, Pawn pawn) - Calculates the i and j after last card was 7

Public void images_fix() - corrects the images above the view buttons

Public void returnToStart(Pawn pawn) - make start buttons visible

Public void moveOnSlider() - represents the movement of the pawns on the slider

Περιέχει και την κλάση PawnListener η οποία κάνει implement την MouseListener.

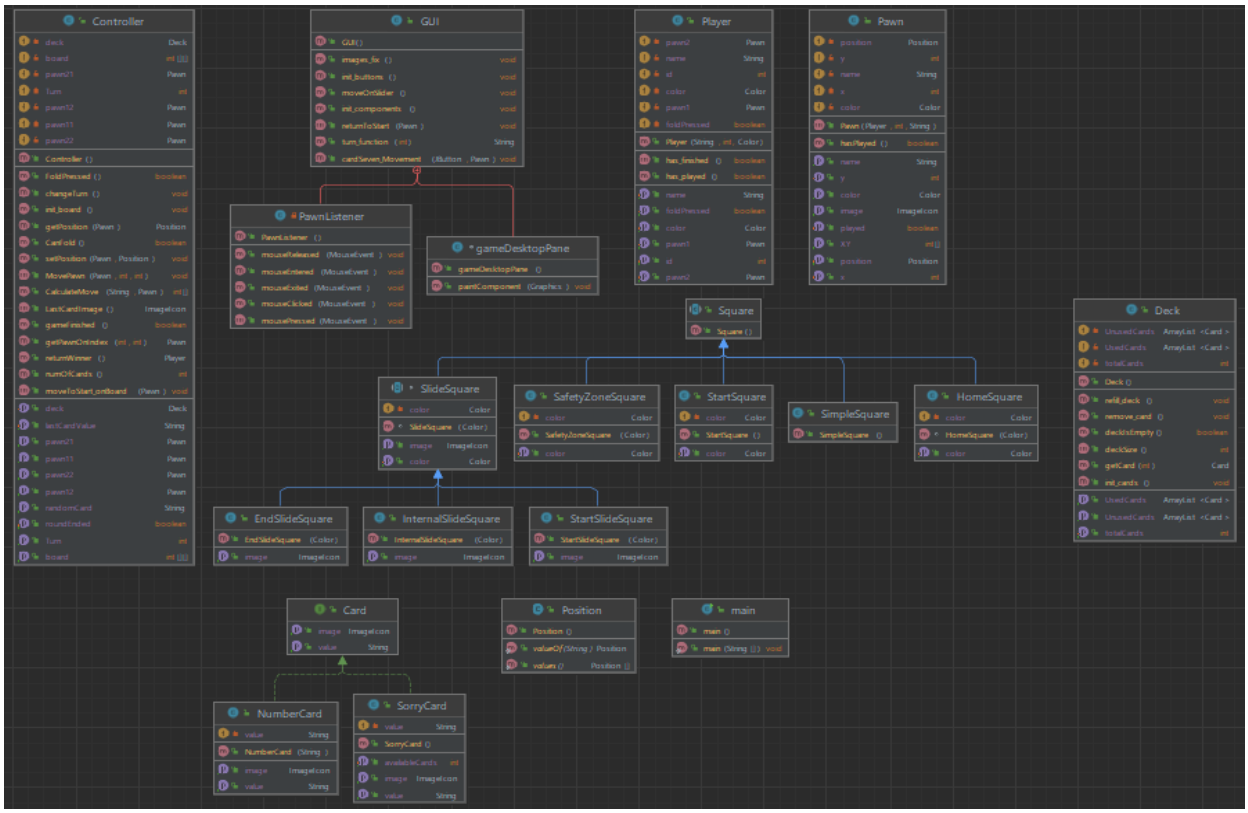
Η PawnListener αποσκοπεί στην ενημέρωση του GUI, ανάλογα με τις επιλογές του παίκτη, με τη χρήση του ποντικιού.

Επίσης, δημιούργησα την κλάση gameDesktopPane που εξειδικεύει την JDeskopPane και έχει ως σκοπό την αρχικοποίηση του background.

Στο GUI σε σχέση με την Α Φάση, έχω προσθέσει τις συναρτήσεις turn_function, cardSeven_Movement, images_fix(), returnToStart, moveOnSlider. Οι συναρτήσεις αυτές με διευκόλυναν να υλοποιήσω το project.

UML της κλάσης GUI καθώς και των υποκλάσεών της

5. Η Αλληλεπίδραση μεταξύ των κλάσεων – Διαγράμματα UML



6. Λειτουργικότητα (Β Φάση)

Τα περισσότερα ζητούμενα της εργασίας έχουν υλοποιηθεί επιτυχώς. Το κομμάτι του κώδικα το οποίο δεν δουλεύει σωστά είναι η αναπαράσταση των πιονιών πάνω στο κουμπί Home. Επίσης ενδεχομένως να λείπουν κάποιοι έλεγχοι στην περίπτωση του fold και στις περιπτώσεις όπου η κάρτα είναι Sorry ή 11.

7. Συμπεράσματα

Η εργασία αυτή αποτέλεσε μια σημαντική εμπειρία για εμένα, καθώς είχα την ευκαιρία να εξοικειωθώ με το μοντέλο MVC.

Το δυσκολότερο κομμάτι που συνάντησα κατα την υλοποίηση του project, ήταν η στήσιμο της γραφικής διεπαφής του project, καθώς η τοποθέτηση των components πάνω στο board ήταν αρκετά χρονοβόρα διαδικασία. Επίσης πολλά components είχαν κάποιες ιδιαιτερότητες σε σχέση με άλλα, οπότε ήθελαν ειδικό χειρισμό. Για παράδειγμα, κατα την υλοποίηση του text box, για φανούν οπτικά οι αλλαγές που γίνονταν μέσα σε αυτό μετά το τέλος του κάθε γύρου, έπρεπε να κάνω reload τα γραφικά του GUI.

Τέλος, κάτι το οποίο θα ήθελα να βελτιώσω στον κώδικα που παρέδωσα, εκτός από τα λειτουργικά θέματα που προανέφερα, θα ήταν ο διαμοιρασμός του κώδικα στο GUI σε περισσότερες συναρτήσεις, καθώς λόγω του όγκου του κώδικα, δυσκολεύτηκα αρκετά σε πολλές περιπτώσεις κατά το debugging.